

PORTADA

Análisis y Diseño de Sistemas II

Semana 7 — Cohesión y Acoplamiento

Universidad de Antioquia · Ingeniería de Sistemas



APERTURA

**Una clase que valida, guarda, envía correos
y genera reportes — todo a la vez**

CONTRA EJEMPLO

La clase que hace todo

```
class GestorPedidos:
    def validar_pedido(self, pedido): ...
    def calcular_impuestos(self, pedido): ...
    def guardar_en_base_de_datos(self, pedido): ...
    def enviar_correo_confirmacion(self, pedido): ...
    def generar_reporte_pdf(self, pedido): ...
    def registrar_en_log_de_auditoria(self, pedido): ...
```

Una sola clase mezcla validación, cálculo de negocio, persistencia, notificación, generación de reportes y auditoría. **Cambiar cómo se envían los correos obliga a tocar la misma clase que calcula impuestos.**

TRANSICIÓN

El problema tiene nombre: baja cohesión, alto acoplamiento

Ayer vimos cómo proteger el estado interno de una clase con encapsulamiento. Hoy damos un paso más: **¿cómo se reparten bien las responsabilidades entre varias clases?** Esa pregunta la responden dos propiedades del diseño: cohesión y acoplamiento.

CONCEPTO

Qué es cohesión

Cohesión mide qué tan relacionadas entre sí están las responsabilidades que agrupa una clase (o un módulo). Una clase tiene **alta cohesión** cuando todos sus métodos y atributos giran alrededor de un único propósito bien definido.

Una clase `CalculadoraDeImpuestos` con métodos que solo calculan impuestos es altamente cohesiva. `GestorPedidos` del ejemplo anterior no lo es: agrupa seis responsabilidades sin relación directa entre sí.

CONCEPTO

Qué es acoplamiento

Acoplamiento mide qué tan dependiente es una clase (o módulo) de los detalles internos de otras. Una clase tiene **bajo acoplamiento** cuando puede cambiar sin obligar a modificar muchas otras clases, y cuando entiende de las demás solo lo mínimo necesario para colaborar con ellas.

Alta cohesión y bajo acoplamiento no son ideas aisladas — son, junto con abstracción y encapsulamiento de ayer, el objetivo central de todo diseño orientado a objetos.

CONCEPTO

Tipos de acoplamiento, de peor a mejor

Tipo	Qué implica
De contenido	Una clase modifica directamente el estado interno de otra (ej. accede a un atributo privado por algún atajo)
Común	Varias clases comparten y modifican la misma variable o estado global
De control	Una clase le indica a otra <i>cómo</i> hacer su trabajo, pasándole banderas o parámetros que alteran su lógica interna
De datos	Una clase solo intercambia los datos estrictamente necesarios a través de parámetros bien definidos — el más deseable

El objetivo de diseño es siempre acercarse al acoplamiento de datos y alejarse del acoplamiento de contenido.

CONCEPTO

Por qué son el objetivo central del diseño OO

- **Alta cohesión** hace que cada clase sea fácil de entender, probar y reutilizar — tiene un solo motivo para cambiar.
- **Bajo acoplamiento** hace que un cambio en una clase no se propague como una reacción en cadena por todo el sistema.
- Juntas, reducen el **costo del cambio** — la métrica que más le importa a un sistema que va a vivir varios años en producción.
- Un sistema con clases muy cohesivas pero muy acopladas entre sí sigue siendo frágil — **hay que perseguir ambas propiedades a la vez**, no una sola.

CONTRA EJEMPLO

Señales de baja cohesión: la clase "todóloga"

Una clase de baja cohesión —conocida coloquialmente como **God Object ("objeto Dios")**— acumula responsabilidades no relacionadas hasta volverse el centro de gravedad de todo el sistema. Señales típicas:

- No se puede describir su propósito en una sola frase sin usar la palabra "y".
- Tiene decenas de métodos que operan sobre subconjuntos distintos de sus atributos.
- Casi cualquier cambio en el sistema termina tocándola.
- Su nombre es genérico: `Manager` , `Utils` , `Helper` , `GestorTodo` .

CONCEPTO

Una pregunta simple para detectarlas

"¿Puedo describir esta clase en una sola frase, sin usar la palabra 'y'?"

Si la respuesta natural es "*esta clase valida pedidos y envía correos y genera reportes*", la clase probablemente debería dividirse en tres: cada "y" es una responsabilidad candidata a su propia clase.

Esta heurística anticipa —sin nombrarlo todavía formalmente— el **Principio de Responsabilidad Única** que verán en la Unidad 5 como parte de SOLID.

CONCEPTO

Refactorizando hacia alta cohesión

```
class ValidadorDePedidos:
    def validar(self, pedido): ...

class CalculadoraDeImpuestos:
    def calcular(self, pedido): ...

class RepositorioDePedidos:
    def guardar(self, pedido): ...

class NotificadorDePedidos:
    def enviar_confirmacion(self, pedido): ...
```

`GestorPedidos` se divide en cuatro clases, cada una con un único motivo para cambiar. **Cambiar el proveedor de correo ahora solo toca `NotificadorDePedidos` .**

CONCEPTO

Cohesión y acoplamiento suelen moverse juntos

Cuando se divide una clase de baja cohesión en varias clases cohesivas, es fácil introducir **acoplamiento innecesario** entre ellas si no se diseña con cuidado quién conoce a quién.

Situación	Diagnóstico
Clase cohesiva, pero conocida y usada por 20 clases distintas	Cohesión alta, acoplamiento alto
Clases pequeñas que constantemente se llaman entre sí en cadena	Cohesión alta, acoplamiento alto
Clase con una sola responsabilidad, usada por pocas clases a través de una interfaz clara	Cohesión alta, acoplamiento bajo — el objetivo

TENSIÓN CONCEPTUAL

Cohesión perfecta también tiene un costo

Dividir una clase hasta el extremo —una clase por cada método— no mejora el diseño: produce **sobre-fragmentación**. El sistema termina con decenas de clases diminutas, difíciles de rastrear, donde entender un solo flujo de negocio exige saltar entre demasiados archivos.

El objetivo no es maximizar el número de clases, sino que cada una tenga un propósito claro y las dependencias entre ellas sean pocas y explícitas. Cohesión y simplicidad de navegación también deben balancearse.



INTERACCIÓN

Identifiquen el tipo de acoplamiento

```
class ReporteVentas:
    def generar(self, tipo_reporte):
        if tipo_reporte == "resumen":
            # ...
        elif tipo_reporte == "detallado":
            # el llamador decide, con una bandera,
            # qué rama de lógica interna ejecutar
```

En el chat: ¿qué tipo de acoplamiento de la tabla de la diapositiva 7 ilustra este ejemplo? ¿Cómo lo rediseñarían para bajarlo a acoplamiento de datos?

5 minutos · respondan en el chat

CONCEPTO

Acoplamiento entre capas

La arquitectura N-capas de la semana 6 es, en el fondo, una estrategia para **controlar el acoplamiento a gran escala**: cada capa (Controller , Service , Repository) solo conoce la interfaz de la capa vecina, nunca su implementación interna.

Las mismas reglas que aplican entre dos clases aplican entre dos capas completas — es el mismo principio de diseño, aplicado en una escala más grande.

CONCEPTO · LO QUE VIENE

El puente hacia GRASP

En la Unidad 5 (semanas 10–13) van a conocer **GRASP (General Responsibility Assignment Software Patterns)**, un conjunto de patrones para decidir qué clase debe asumir cada responsabilidad. Dos de esos patrones se llaman literalmente **Low Coupling** y **High Cohesion**.

Todo lo visto hoy es la intuición que esos patrones formalizan más adelante — hoy aprenden a reconocerlas a simple vista; ahí aprenderán reglas explícitas para decidir dónde poner cada responsabilidad.

SÍNTESIS

Ideas clave de hoy

1. **Cohesión:** qué tan relacionadas están las responsabilidades dentro de una misma clase.
2. **Acoplamiento:** qué tan dependiente es una clase de los detalles internos de otras.
3. Cuatro tipos de acoplamiento, de peor a mejor: de contenido, común, de control, de datos.
4. Una clase "todóloga" (God Object) es la señal más clara de baja cohesión.
5. Alta cohesión y bajo acoplamiento deben perseguirse **juntas** — y anticipan los patrones GRASP de la Unidad 5.



CIERRE

Para la próxima sesión

Vuelvan a revisar su modelo de dominio: ¿hay alguna clase que no puedan describir en una frase sin usar "y"? Anótenla — la vamos a usar como insumo en próximas sesiones de diseño.

El viernes cerramos la semana con herencia, polimorfismo e interfaces — los mecanismos que permiten reutilizar y especializar comportamiento entre clases relacionadas, sin sacrificar todo lo visto esta semana.

